Adaptive Interface Elements Generation – 2D Case

Tecgraf PUC-RIO

Last update: 25 – February - 2026

Objective

- Design the operators and algorithm to automatically insert interface elements based on the input of a set of edges (faces in 3D).

- Both, algorithm and operators, should be compatible to IBVHM, an index-based data structure.

Approach

1. Design a simple and small set of operator capable to perform the interface elements' insertion.

2. Solve the implementation in the operators level.

3. Design an algorithm that traversal the input edges and uses the operator to create the interface elements.

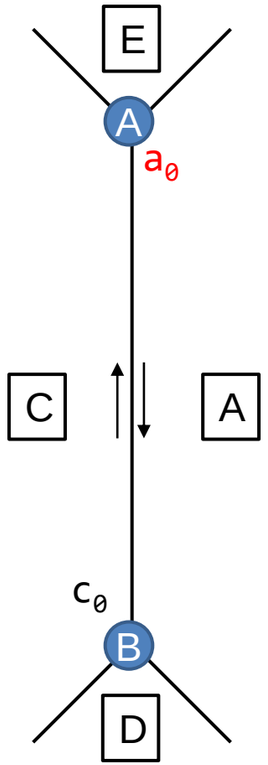
4. Exhaustively test and validate the implementation in 2D.

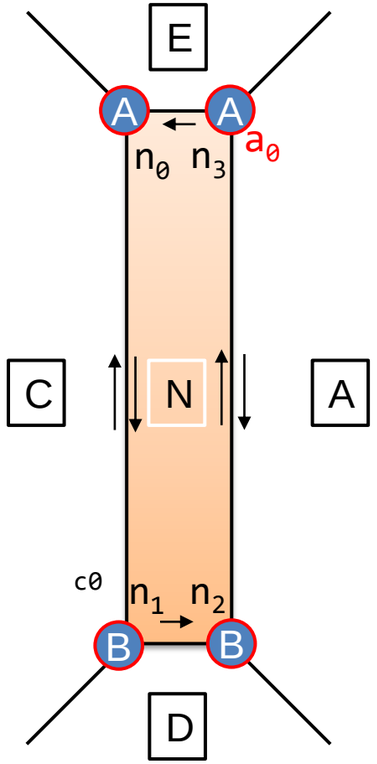
5. Transport the implementation and test it in GeMA.

6. Implement it for 3D elements.

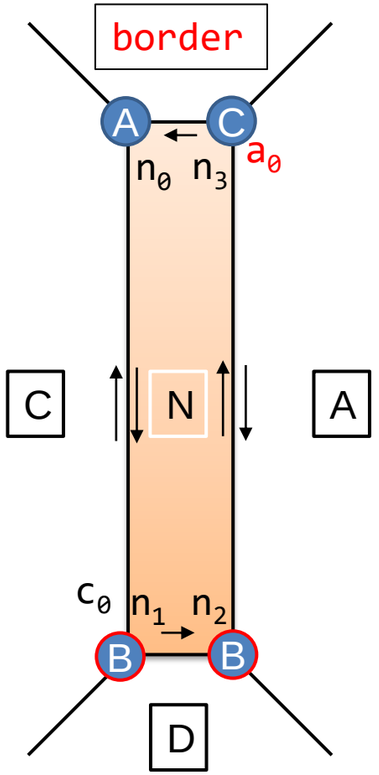
Operators: Canonical

canonical(he = a_0)

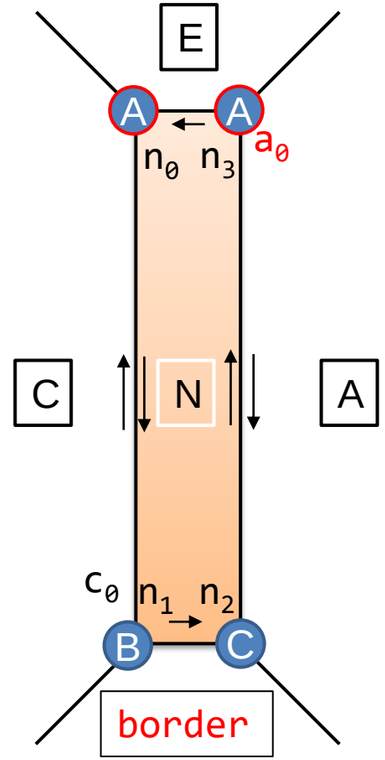




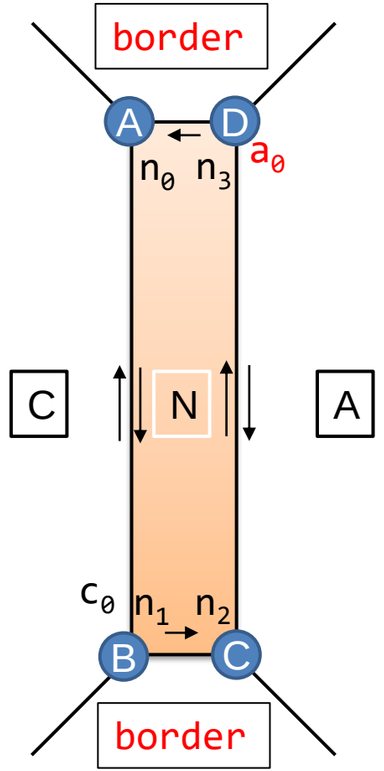
n_1 = collapsed in B.
n_3 = collapsed in A.



n_1 = collapsed in B.
n_3 = border



n_1 = border.
n_3 = collapsed in A.



n_1 = border.
n_3 = border.

Operators: Canonical

- The **Canonical Operator** inserts a **degenerated** interface element in an edge when the vertices do not belong to any interface element yet.

- The operator works like an **initialization** for the vertex to be prepared for other operations. Also, this degeneration is important for the **transitions**.

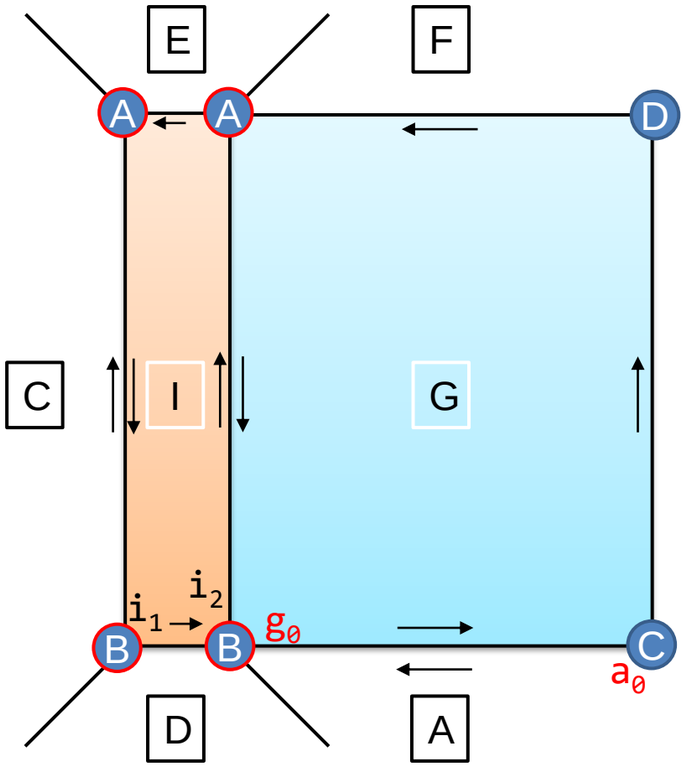
- For each vertex that respects this condition, a **collapsed edge will be created**. The internal structural for the element still working, but the **opposite half-edge** for these edges **is not defined**.

- When both vertices do respect this condition, the resulting interface element will be from the form ABBA with two edges collapsed in AA and BB.

- **The condition is not valid for borders**. In these cases, a new geometric vertex will always be created.

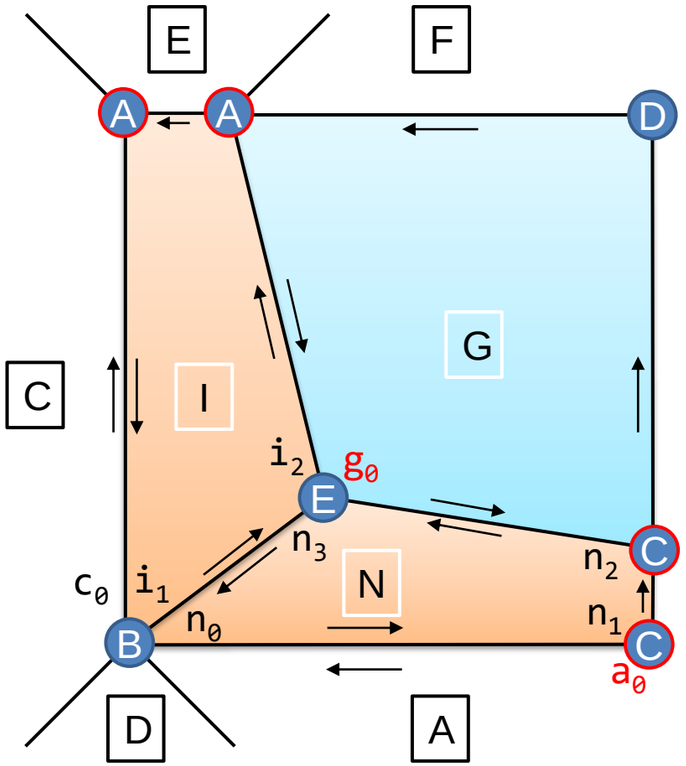
Operators: Split Element

splitElement(he = a_0)



i_1 = collapsed in B.

i_2 = he selected to reindex the element I with the new vertex E.



n_1 = collapsed in C.

This operation is only valid if a collapsed vertex* shares an outcome half-edge with one of the faces shared by the edge being split.

i_2 is a half-edge from a collapsed vertex representing an edge shared with the face G. G has the edge represented by g_0 and a_0, so i_2 can be used as the new vertex E to build the new element.

* Collapsed vertex: a vertex from a collapsed edge.

Operators: Split Element

- The **Split Element Operator** should be used in the following conditions:
  - **Available Vertex:** the edge selected to inset the interface element should have a collapsed edge, which means a duplicated = available vertex.
  - **Shared Element:** the income/outcome edge from the available vertex should share an element with the edge selected to insert the interface element.

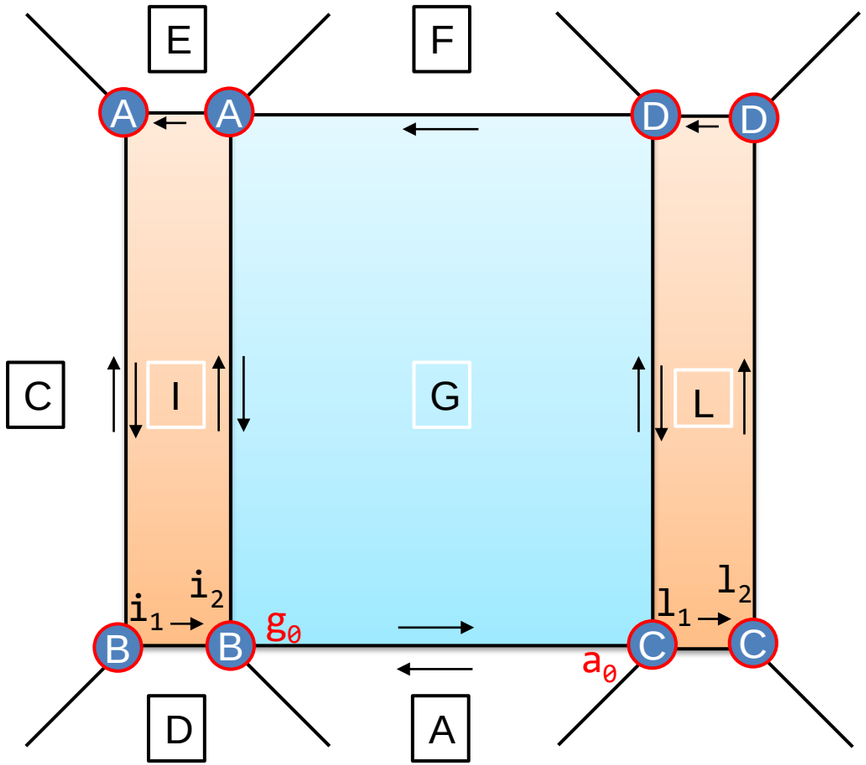
- Once the conditions are verified, a new interface element is inserted in the following way:
  - **Available Vertex:** a new geometric vertex is created and assigned to the identified available vertex.
  - **Former Interface Element:** the collapsed edge in the former interface element now determines the frontier with the brand new interface element.

- This operator replaces the algorithm that rotates around a vertex splitting the faces when the edges are selected to insert interface elements. The difference is that no order is required now.

- This operator only defines what happens with the vertex that follows the defined conditions.

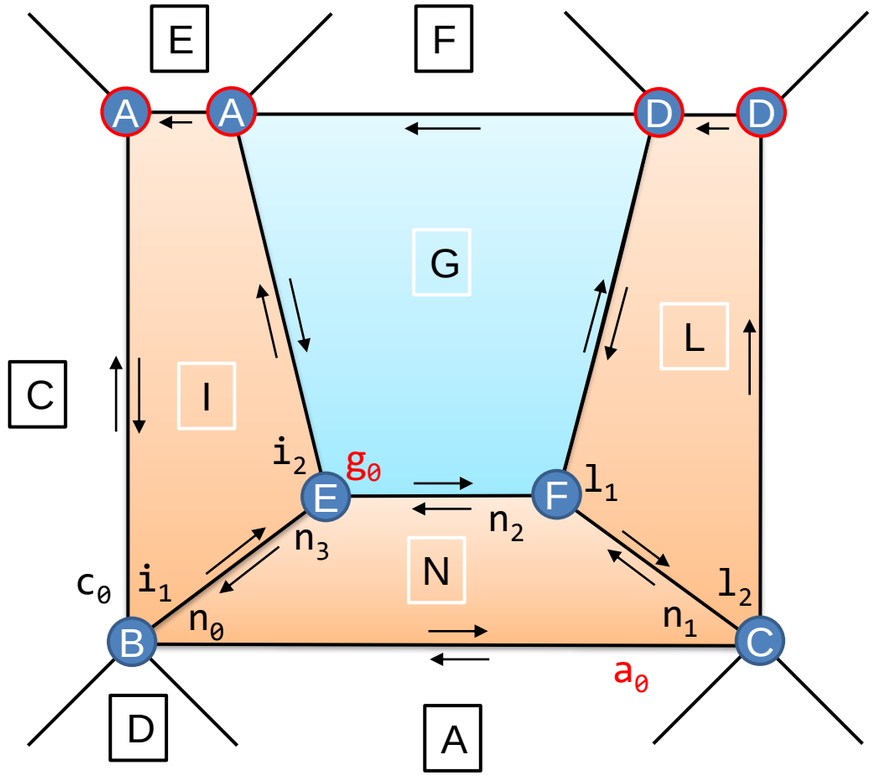
Operators: Split Element

splitElement(he = a_0)



i_1 = collapsed in B.
l_1 = collapsed in C

i_2, l_1 = half-edges selected to reindex the element I with the new vertex E and element L with the new vertex F.

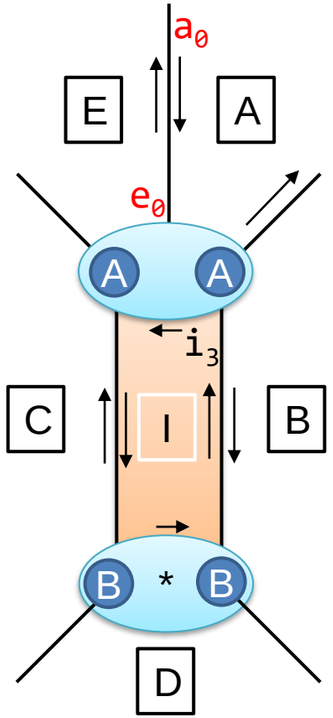


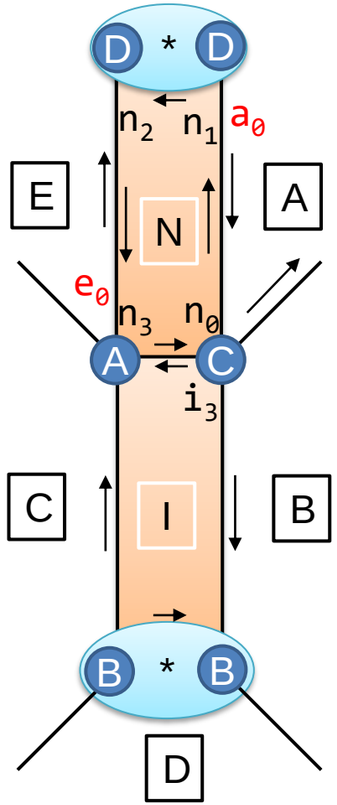
Collapsed in A and D

Same logic, but now applied at the two edge's vertex, once both were collapsed.

Operators: Expand Collapsed Edge

expandCollapsedEdge(he=a_0)





When the previous rule fails to find a face in common, we are in the case of opening an edge directly.

The main challenge here is how to determine which connections should go for each vertex. There are two main approaches:

1) Topological: Rotate around the vertex using the edge being opened to determine the division. However, this operation seems not be directly converted to 3D operation. The 2D rotation is based on the cross product operation, that is not defined for 3D (the operation should be in 4d).
2) Geometric: An edge (or face) separates the world in two, this could be used to determine which edges (faces) should go to each vertex.

Decision: we are going to implement it based on the topological approach because it seems to be safer to convert to 3D. We should avoid twists or operations based on angle. We need to perform a traversal around the vertex starting from both element's side.

* = can be collapsed or not.

Operators: Expand Collapsed Edge

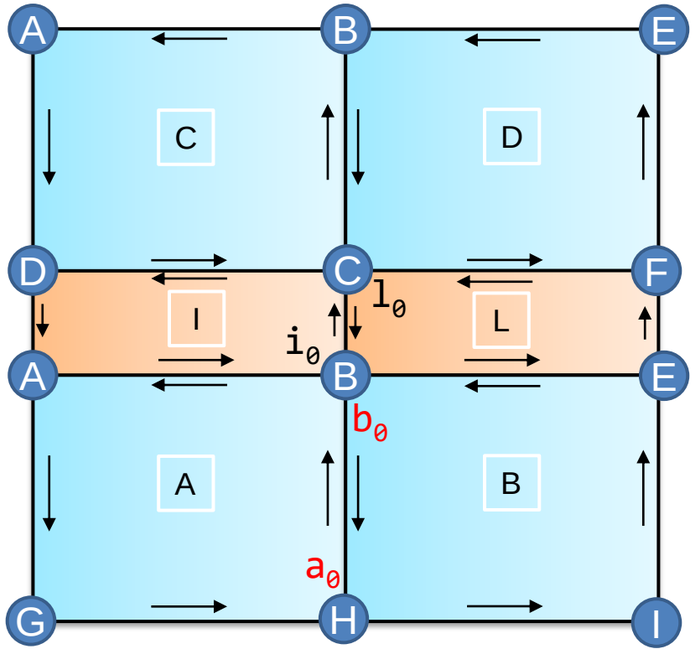
- The **Expand Collapsed Edge Operator** takes place when the former operator condition fails to find a shared element, i.e.:
  - **Available Vertex:** the edge selected to inset the interface element should have a collapsed edge, which means a duplicated = available vertex.
  - **No Shared Element:** there is no shared element between the edge selected and the income/outcome edge from the available vertex.

- The sensible part of this operator is that the edges connected to the available vertex should be divided in two groups and each group reconnected to one of the two new vertices (the detected available vertex and the created vertex).
  - The algorithm should be designed in a way that can be translate to 3D with the minimum effort.

Operators: Hole

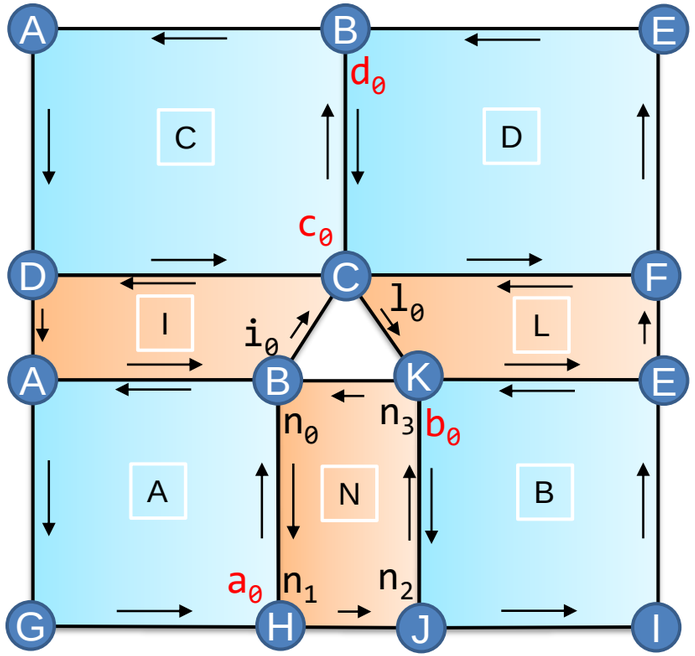
`hole(he = a0)`

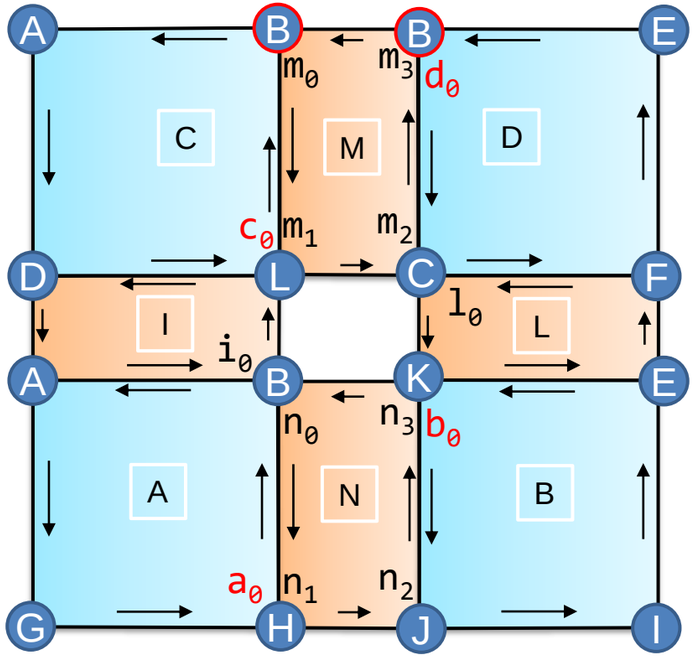
`hole(he = b0)`



`hole(he = c0)`

`hole(he = d0)`





When the vertex has no duplication, a hole should be inserted. The incident vertex (B and C) is always duplicated. If there is no hole yet, the edge in the interface element is also duplicated for the new two vertices. The elements are reindexed considering their respective half-edges. When a hole is already present, it is necessary to classify the edges connecting to vertex into two groups and add them to the respective vertices. It seems that the procedure to separate them here is very similar to the previous operator. Whenever a new edge is opened, the hole increases in number of vertices by one.

Operators: Hole

- The **Hole Operator** should be applied when the vertex is already part of an interface element, but there is no available vertex to be used. In this cases, a vertex will be duplicated, no collapsed edge will be created and a hole will be inserted.

- The vertex duplication creates a new edge. The both edge's vertices should be handled as in **Expand Collapsed Edge Operator**, i.e., the connected edges should be divided into two groups and these groups should be connected to the resulting vertices.

Operators: Hole

Although the process splits an edge creating a new element, the decision about the right procedure should be taken considering the vertices. There are four operation that can be executed depending on the vertex V condition:

1) **A vertex is not part of any interface element:** this is the first operation involving V. It should be duplicated by the **Canonical Operator**. If the vertex V is internal to the mesh, the inserted element will have a collapsed in V. On the other hand, if V is on the mesh border, a new vertex should be created in the geometry and used in the new interface element. In this case, no collapsed edge is necessary.

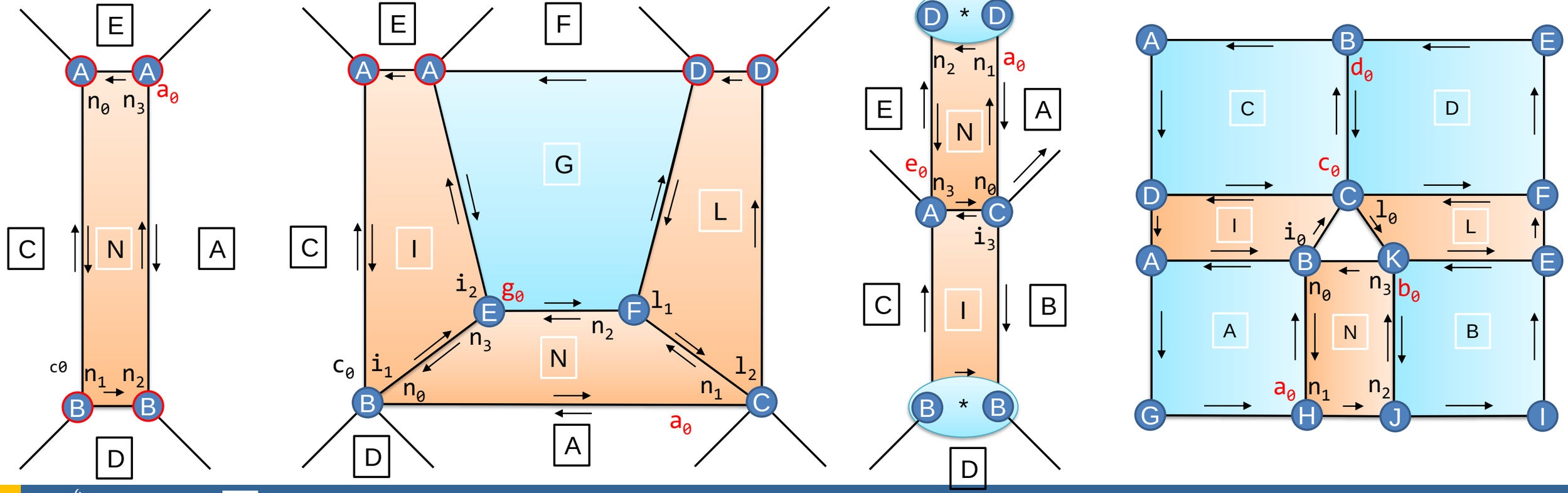
2) **A vertex is part of an interface element and it is duplicated:**
   a) **There is an outcome edge from the vertex that shares an element with the edge being spitted:** the duplicated vertex should be placed inside the shared element by the **Split Element Operator**. The half-edge from the identified vertex should be used to index the new created vertex in shared and new elements.

   b) **There is not face being shared:** with no face in common, it should be applied the **Expand Collapsed Edge Operator**, which is an extension of **Canonical Operator**. The collapsed edge should be opened and the edges connected to V should be divided in two groups to be reconnected to each one of the resulting two vertices.

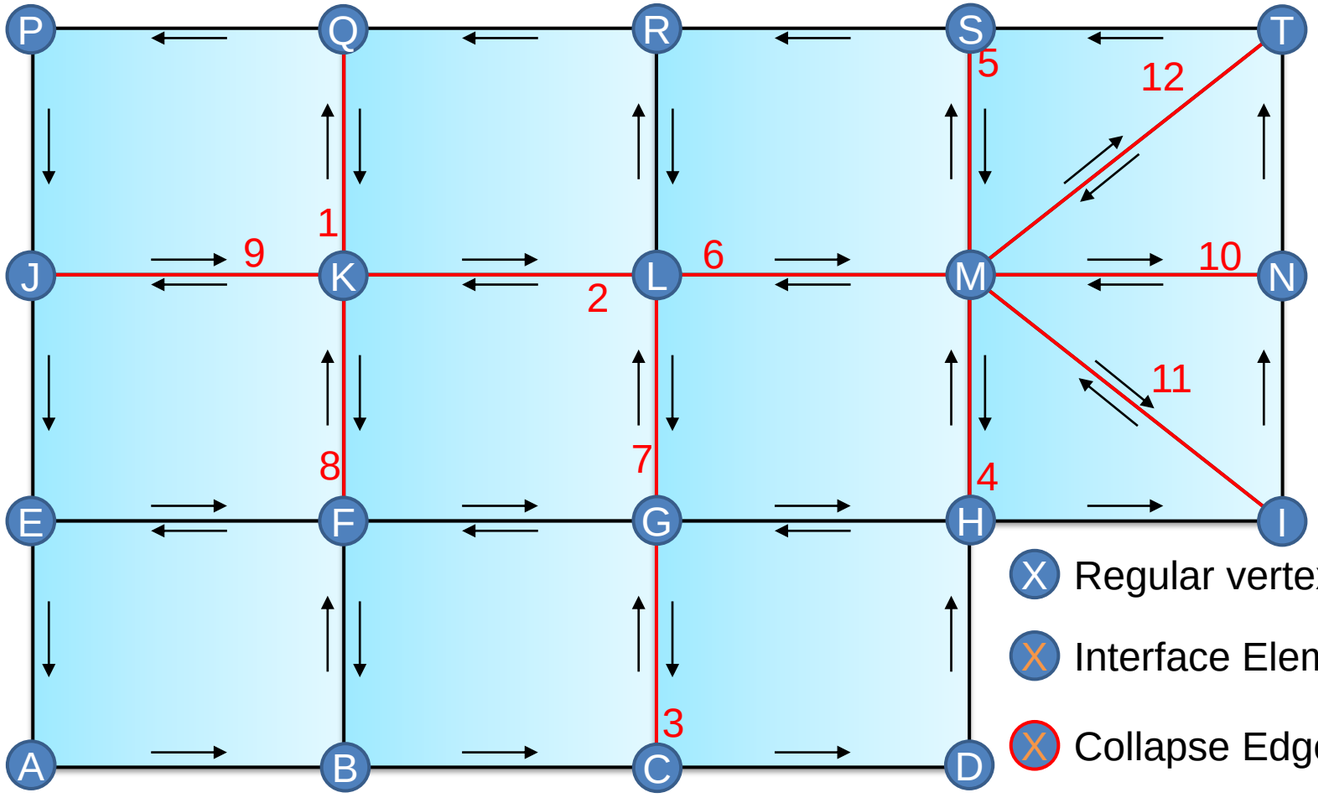
3) **The vertex is part of an interface element, but is not duplicated:** this condition will open a hole, inserted by the **Hole Operator**. V will be duplicated not only the topology, but also in the geometry. If V is not part of a hole yet, the edge in the interface element is also duplicated for the new two vertices. This duplication will open a hole in a triangular shape. The elements are reindexed considering their respective half-edges. When a hole is already present, it is necessary to classify the edges connecting to vertex into two groups and add them to the respective vertices after the vertex duplication. It seems that the procedure to separate them here is very similar to the previous operator. Whenever a new edge is opened, the hole increases in number of vertices by one.

The problem is on the vertices

- Although the problem is **splitting edges (or faces)** and **creating new elements**, all the operators are designed to work on to vertices. This property is desired in order to be easily translated to 3D.
- **What happens in the middle?** The two edges in the middle requires the same updates for all operators:



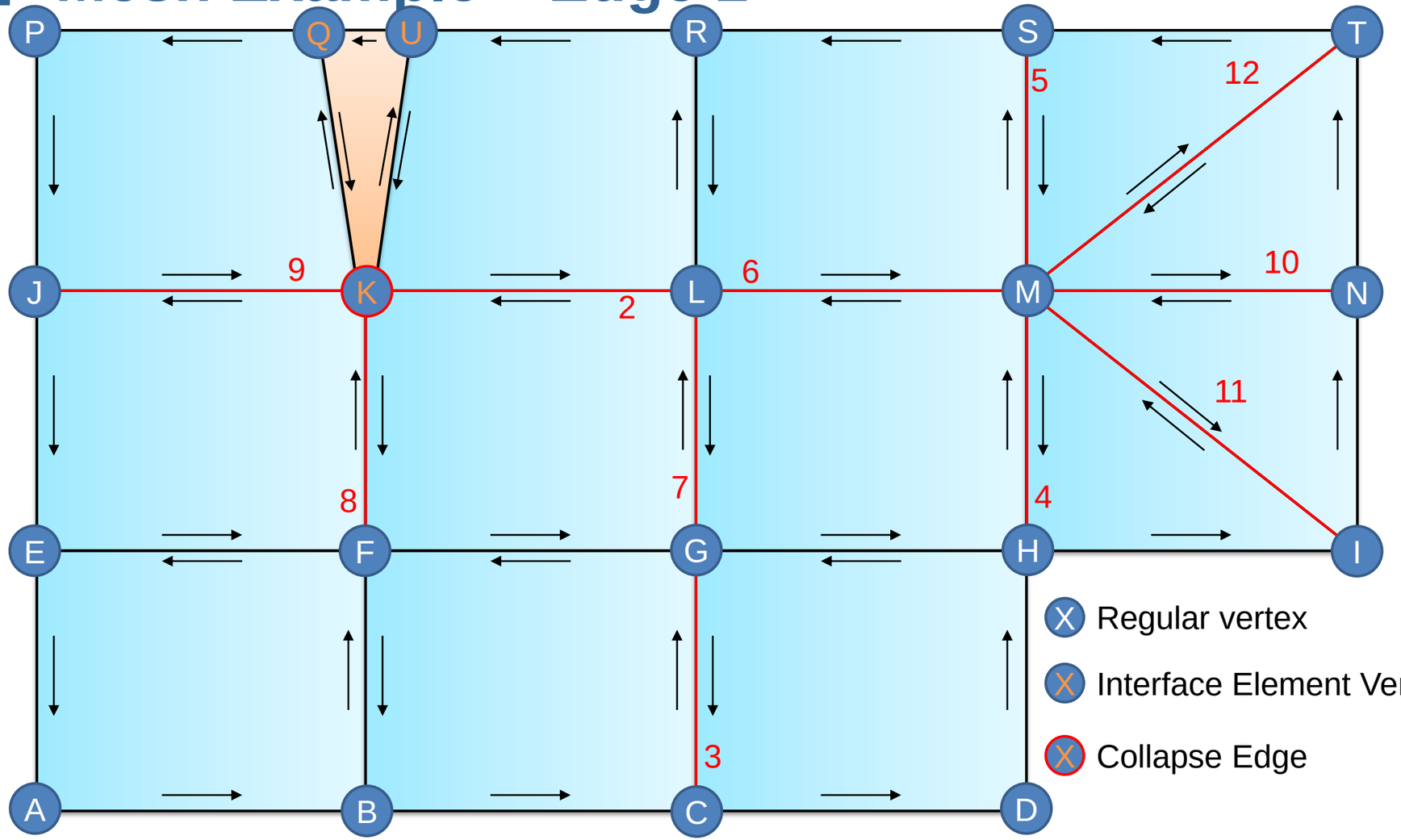
Mesh Example



Mesh example: the selected edges to create interface elements are in red and the order is determined by the numbers for each edge.

Edge 1:
Q: Canonical and Border.
K: Canonical and Internal.

Mesh Example – Edge 1



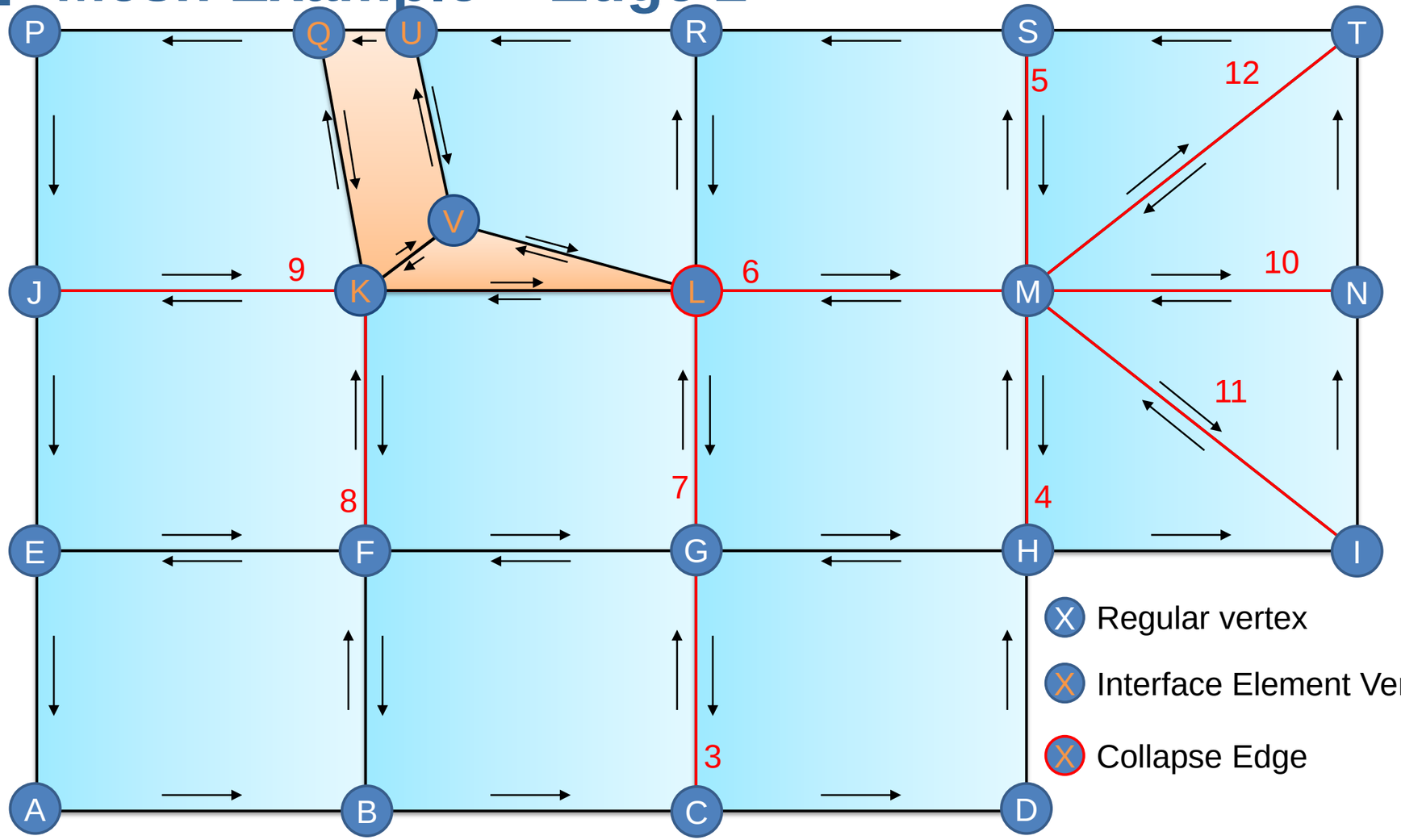
Edge 2:
K: SplitElement with **KLRU** shared.
L: Canonical and Internal.

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 2



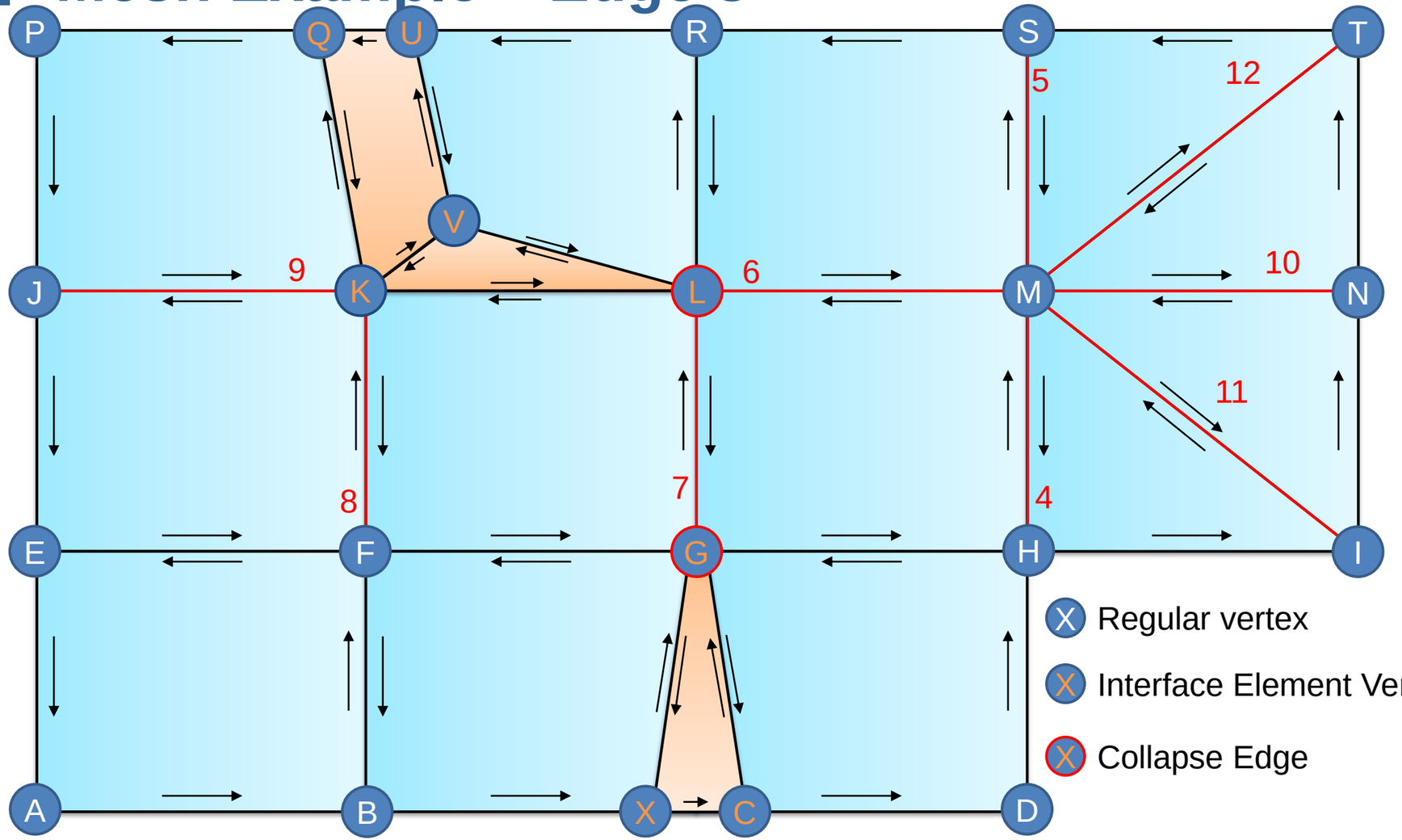
Edge 3:
G: Canonical and Internal.
C: Canonical and Border.

Regular vertex

Interface Element Vertex

Collapse Edge

Mesh Example – Edge 3



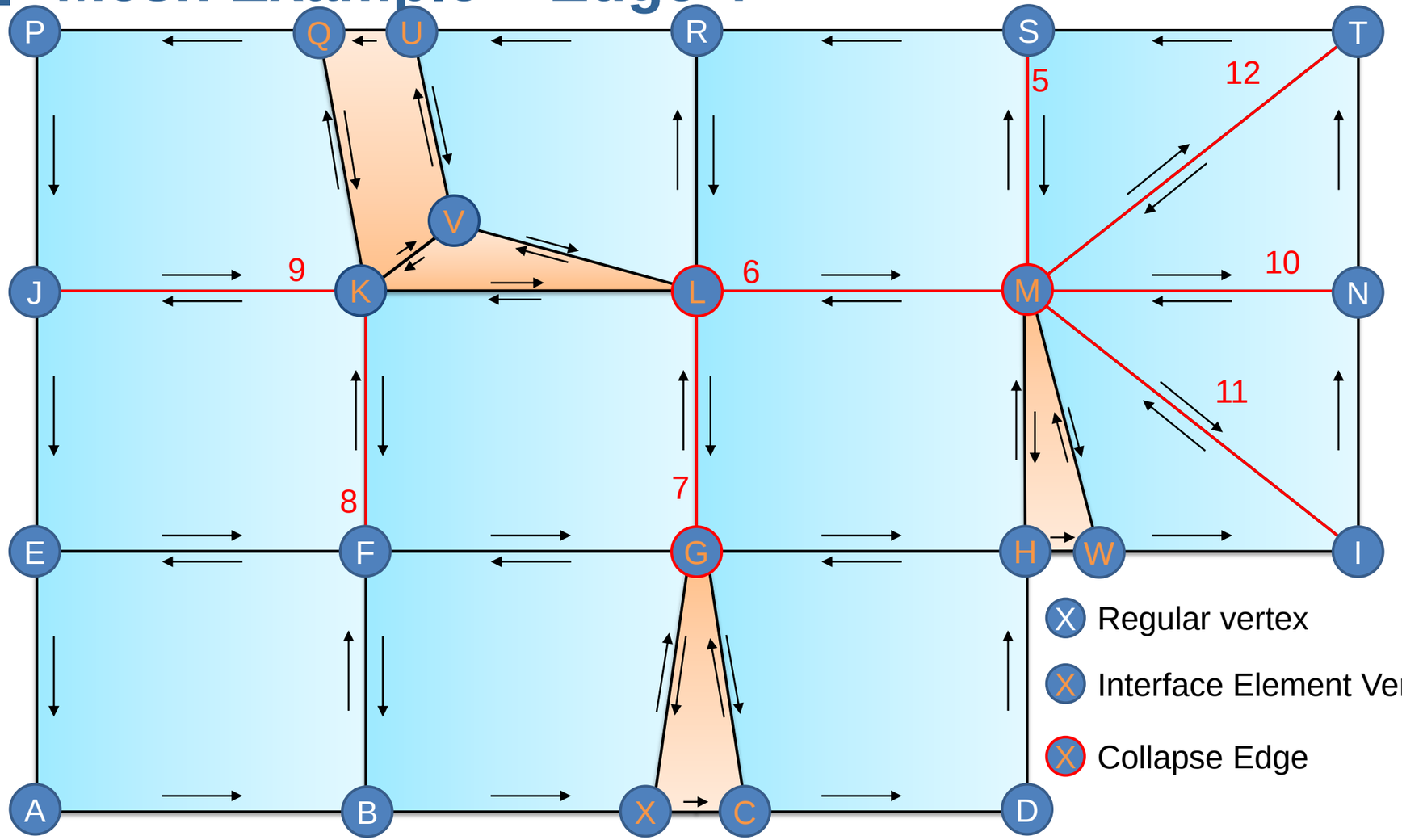
Edge 4:
M: Canonical and Internal.
H: Canonical and Border.

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 4



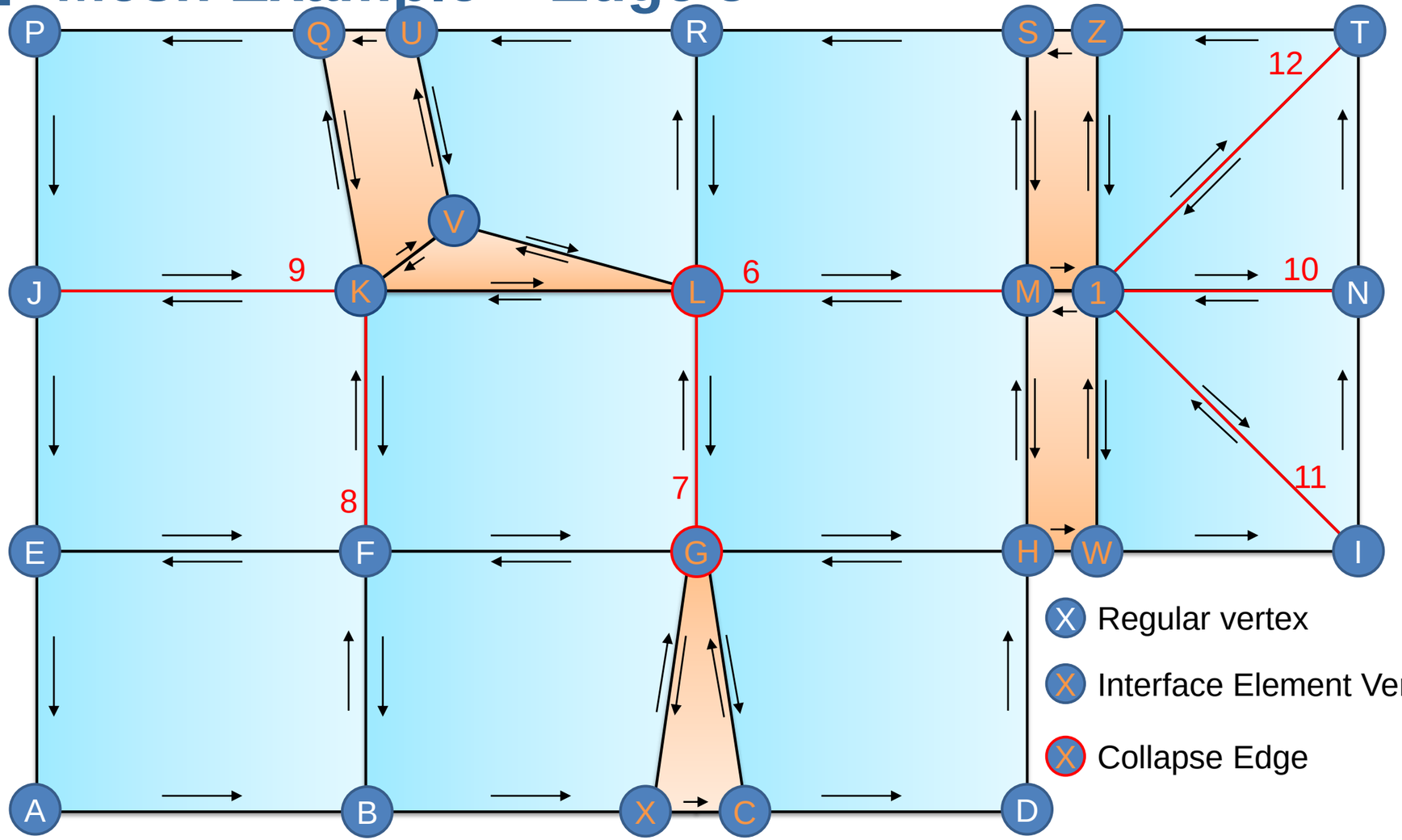
Edge 5:
S: Canonical and Border.
M: Expand Collapsed Edge

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 5



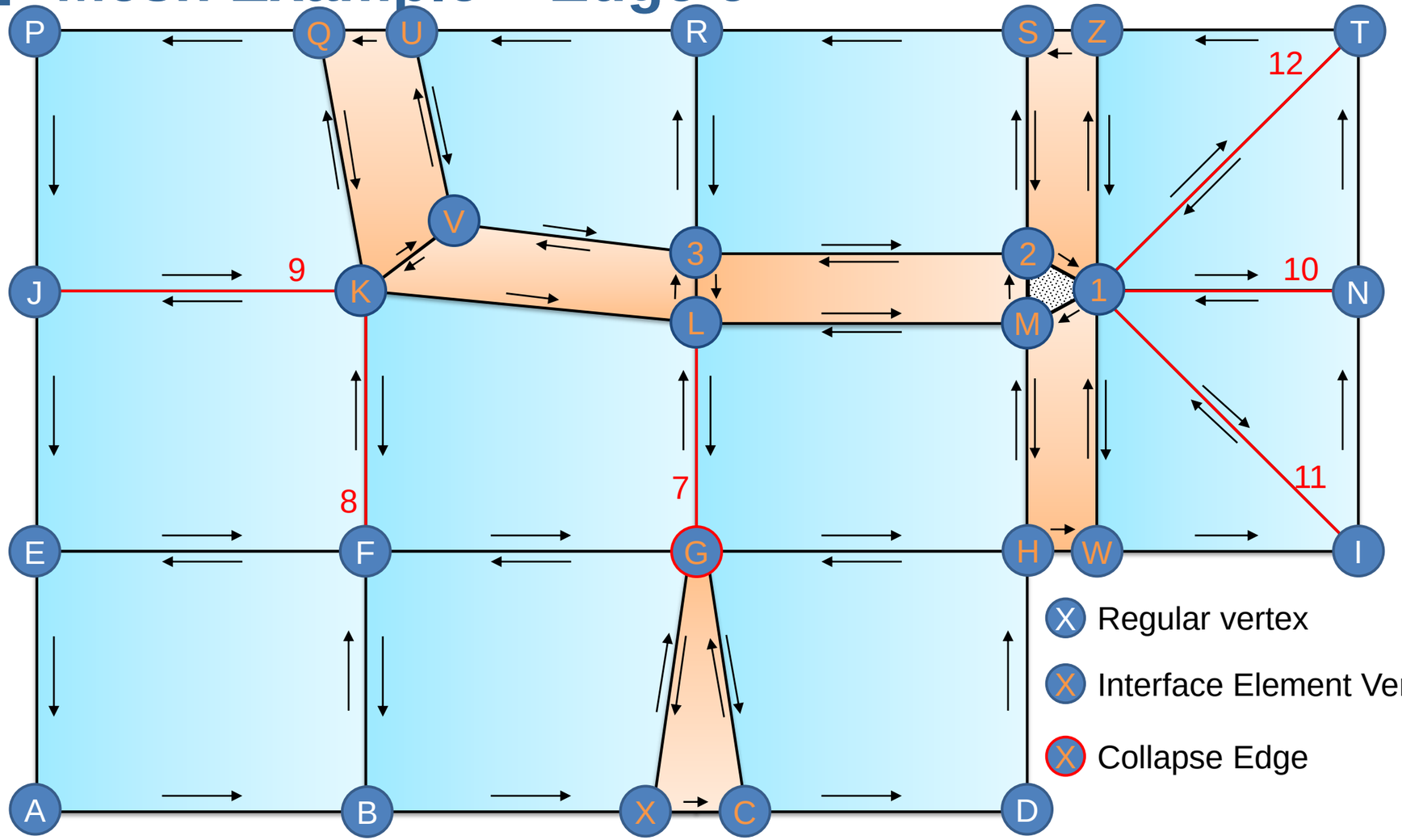
Edge 6:
L: Expand Collapsed Edge
M: Hole

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 6



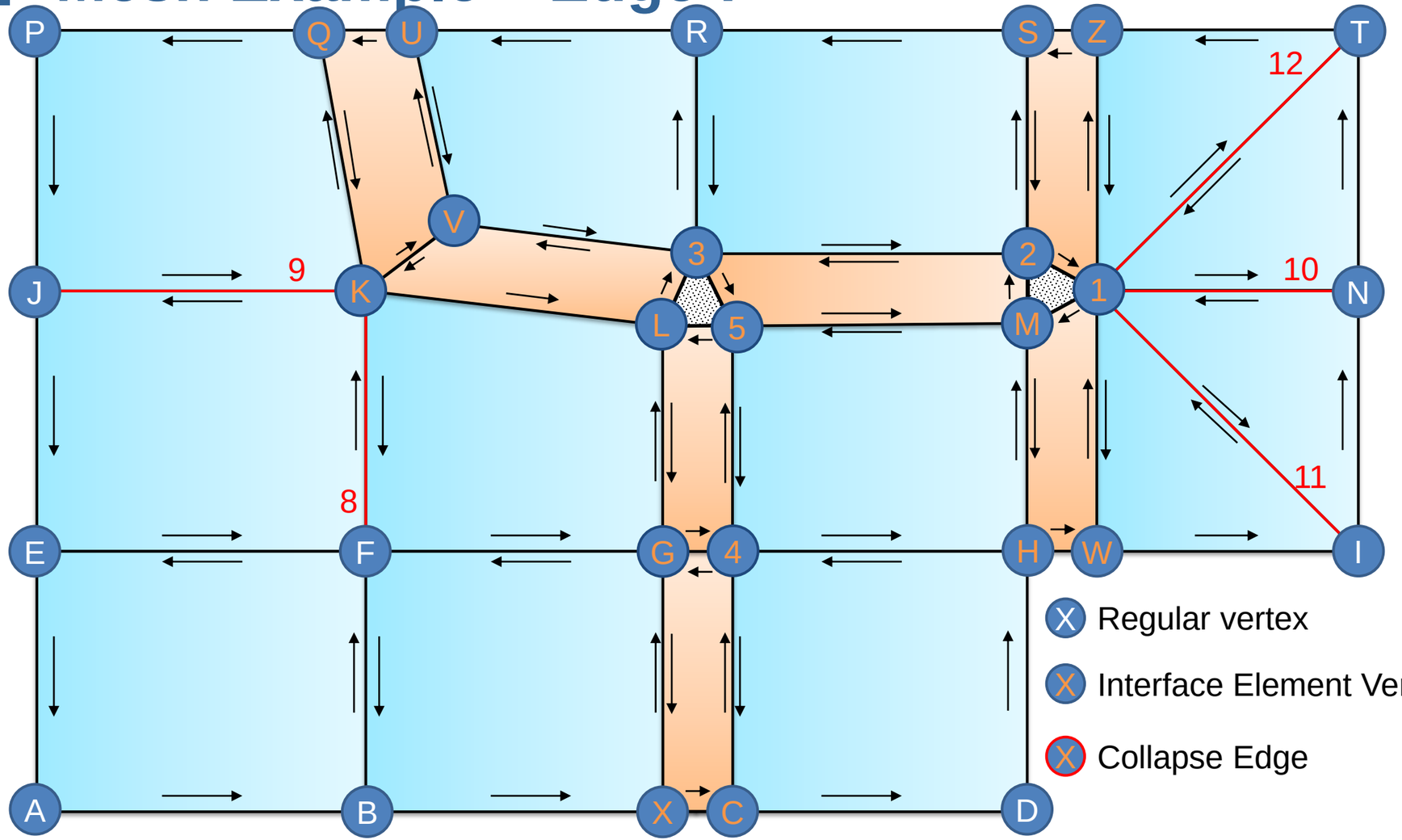
Edge 7:
L: Hole
G: Expand Collapsed Edge

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 7



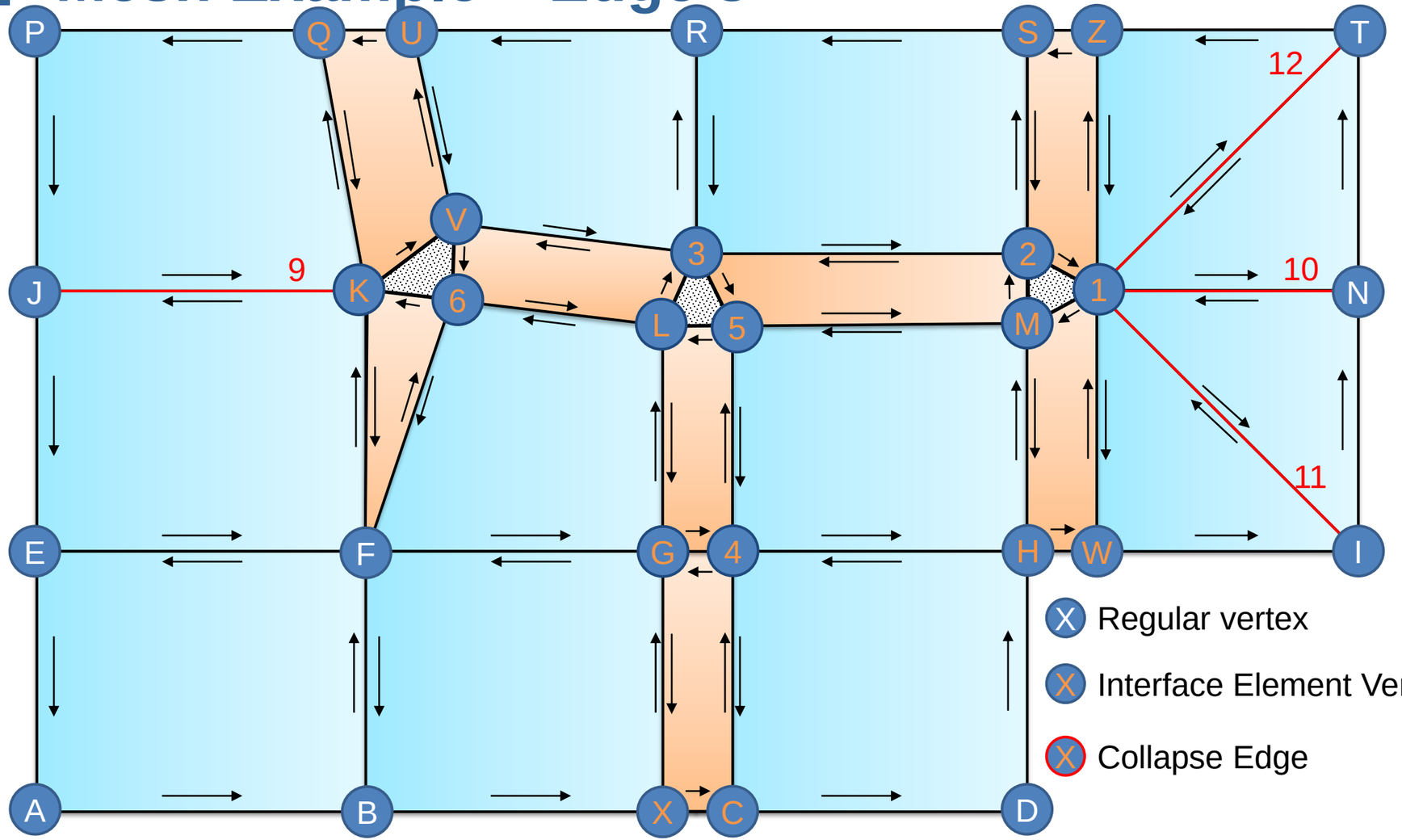
Edge 8:
K: Hole
F: Canonical and Internal.

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 8



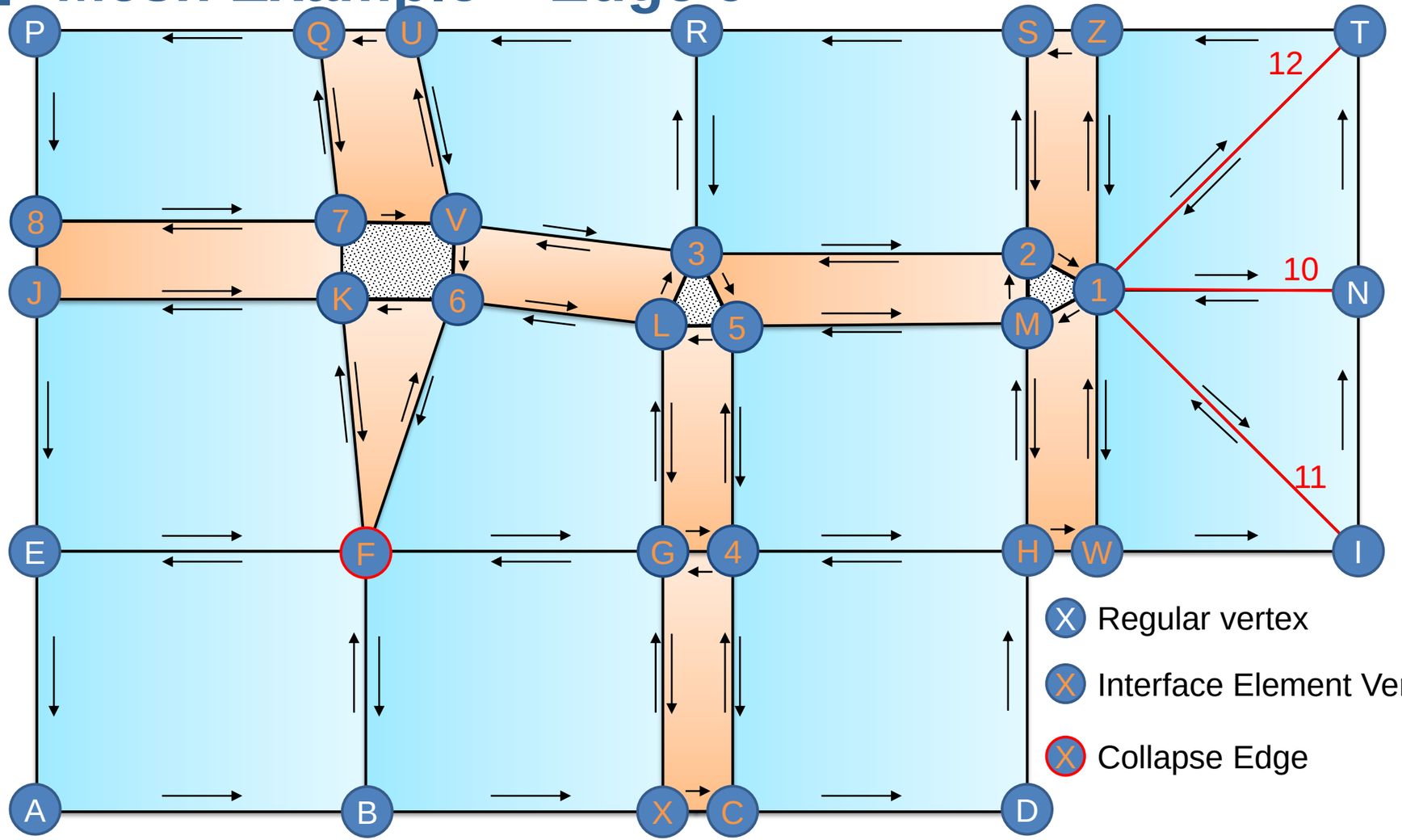
Edge 9:
K: Hole
J: Canonical and Border.

Regular vertex

Interface Element Vertex

Collapse Edge

Mesh Example – Edge 9



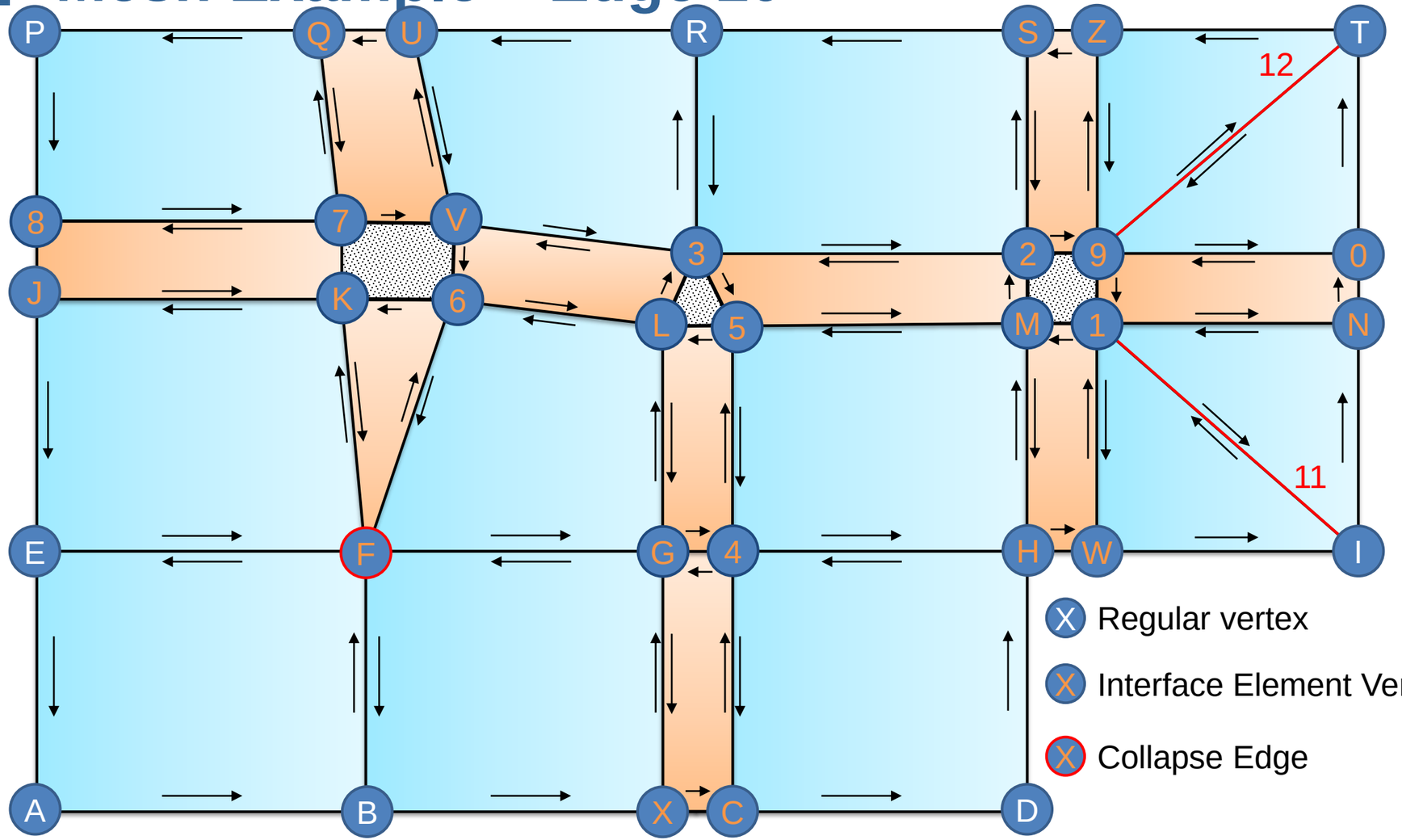
Edge 10:
1: Hole
N: Canonical and Border.

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 10



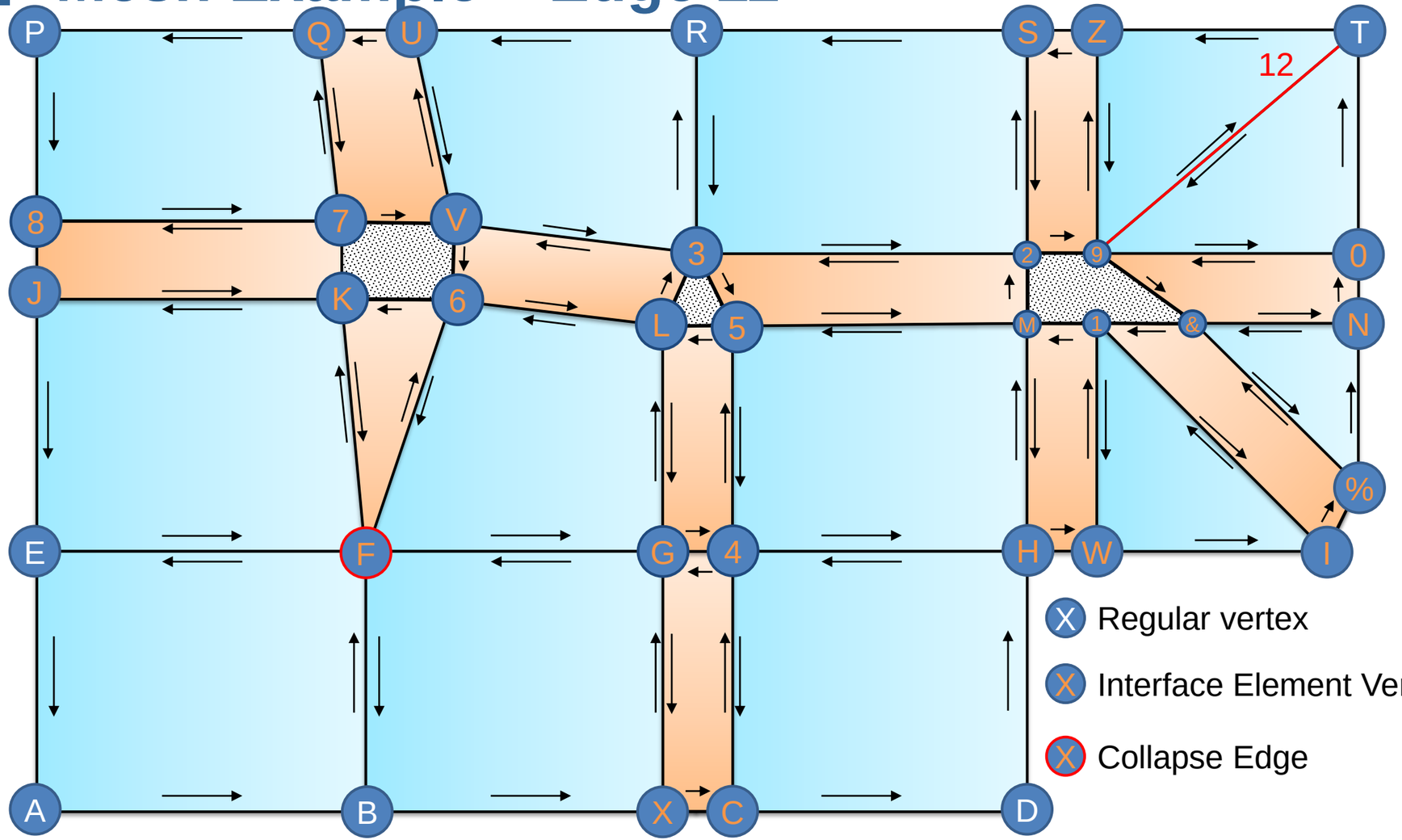
Edge 11:
1: Hole
I: Canonical and Border.

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 11



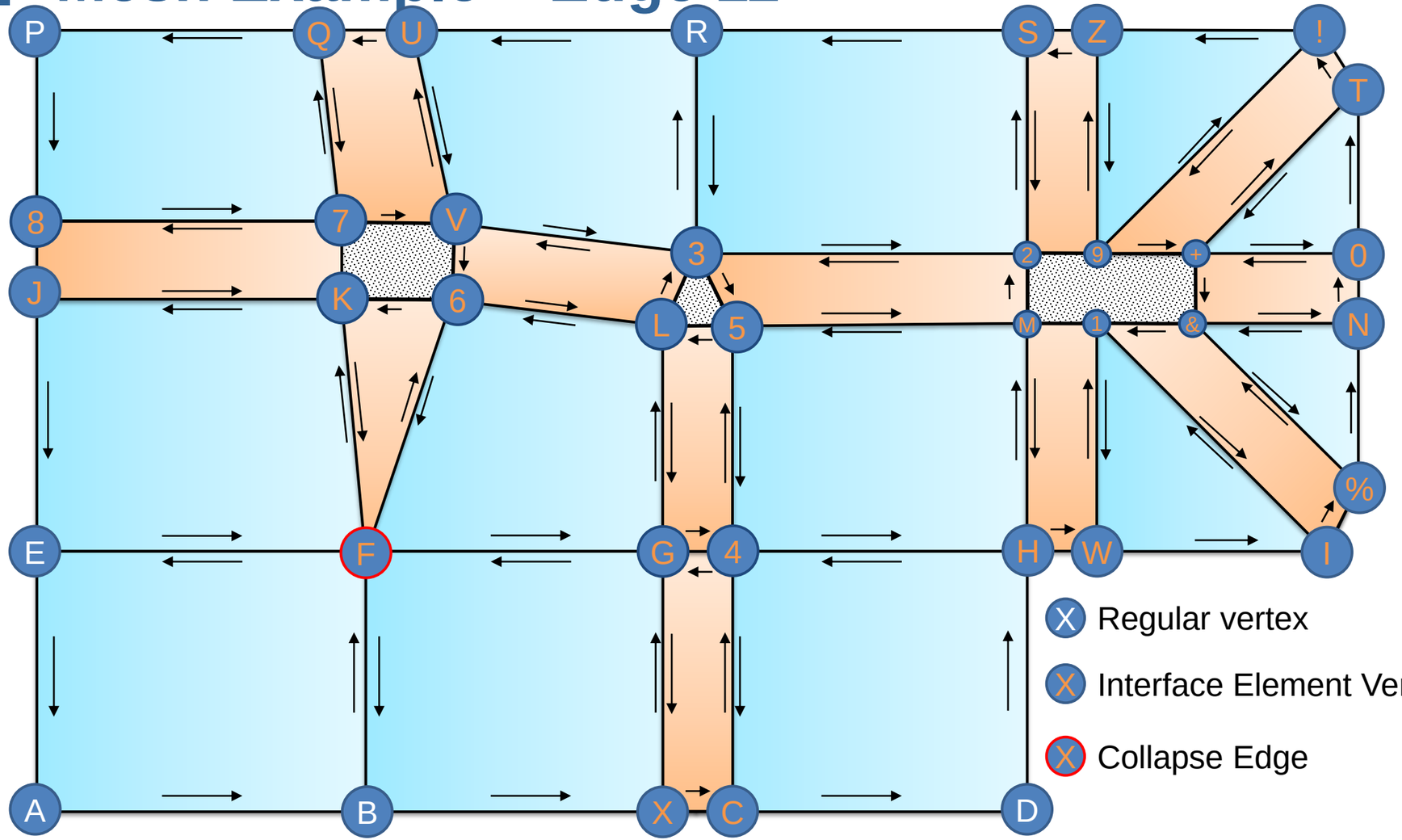
Edge 12:
9: Hole
T: Canonical and Border.

X Regular vertex

X Interface Element Vertex

X Collapse Edge

Mesh Example – Edge 11



X Regular vertex

X Interface Element Vertex

X Collapse Edge